MAUÁ	PREENCHA SEUS DADOS NOS CAMPOS COM X	
Disc.: **ECM251 – LINGUAGEM DE PROGRAMAÇÃO I**		
Curso: **ENGENHARIA DA COMPUTAÇÃO**		

Aluno: X				
Curso: X	Série: X	Período: X	RG: X	
São Caetano do Sul, 19 de novembro de 2024.			RA: X	
Assinatura:			Nota:	

Instruções da Prova

1. Esta prova é individual e prática, devendo ser realizada nos computadores do IMT, sendo permitido ao aluno, se assim desejar, utilizar seu próprio computador, sob sua inteira responsabilidade;
2. Não poderá haver acesso à Internet, sob nenhuma circunstância, exceto à plataforma CanvasLMS do próprio aluno e, mesmo assim, somente em duas etapas: para receber ("baixar") as questões de sua prova na máquina em uso; e para entregar ("subir") as resoluções das questões de sua prova no devido local de entrega, na mesma plataforma.

   IMPORTANTE: O aluno deverá informar ao professor quando fará os dois acessos permitidos a ele ao CanvasLMS, pela Internet;
3. Poderá haver consulta a qualquer material do próprio aluno, seja ele físico (livros, artigos, material de aula etc.) ou digital (livros, artigos, material de aula, exercícios, resoluções etc.), desde que esse material esteja previamente armazenado em seu computador ou em qualquer dispositivo de armazenamento externo (*pendrive*, *hd* externo etc.).

   IMPORTANTE: Não será permitido o acesso pela Internet a pastas compartilhadas (*Google Drive*, *OneDrive* etc.), nem a repositórios virtuais (*GitHub*, *GitLab*, *BitBucket* etc.), mesmo sendo de posse e administração do próprio aluno;
4. O aluno deverá responder às questões da prova no próprio arquivo da prova (*.docx*) devidamente identificado (RA e Nome completo do Aluno) e quando for o caso, gerando os códigos necessários, somente na linguagem de programação JAVA e utilizando sempre e somente o paradigma da Programação Orientada à Objetos visto nas aulas;
5. Para realizar a entrega da prova na plataforma CanvasLMS, o aluno deverá gerar e entregar um único arquivo compactado (*.rar* ou *.zip*), tendo seu RA e seu NOME completo como nome desse arquivo. Nesse arquivo compactado o aluno deverá fornecer, obrigatoriamente, os seus dados acadêmicos preenchidos na prova, bem como a resolução das questões nela solicitadas (*.docx*), além dos arquivos e códigos gerados em suas resoluções, uma pasta para cada questão, contendo as suas classes e demais arquivos que possibilitem sua posterior execução pelo professor, durante a resolução.

   IMPORTANTE: Na correção, para executar o código gerado pelo aluno em sua prova, o professor seguirá exatamente as instruções fornecidas pelos alunos e contidas nas resoluções das provas! Caso não obtenha sucesso, a questão será considerada errada.
6. Não poderá haver troca de informações, nem de materiais, sejam físicos ou digitais, entre os alunos durante a prova;
7. Não é permitido ao estudante se ausentar da sala antes da entrega da prova;
8. Celulares, *Smartphones*, *Smartwatches* e outros equipamentos eletrônicos devem permanecer desligados enquanto o estudante estiver na sala;
9. O tempo limite para realização da prova é de **90** minutos;
10. Mantenha sobre a carteira apenas um documento com foto, caneta, lápis e borracha;
11. O entendimento das questões faz parte da avaliação;
12. O tempo mínimo de permanência na sala é de **30** minutos;
13. O estudante que chegar atrasado em até **30** minutos do início da prova poderá fazê-la.

<u>Questão 1</u>: (*1 ponto*)

Na Engenharia de *Software*, a Arquitetura Cliente-Servidor é um modelo fundamental que descreve a distribuição de funções e responsabilidades em um sistema de *software*. Nesse modelo, o sistema é dividido em duas partes principais, Cliente e Servidor, que interagem entre si, via rede de comunicação e seus protocolos de comunicação, para realizar tarefas específicas através de transações e interações.

Sendo assim, com base na referida bibliografia e no material de aula utilizados, avalie as afirmações a seguir:

I. As principais características da Arquitetura Cliente-Servidor são: cliente; servidor; banco de dados; comunicação; distribuição de tarefas; e escalabilidade.

II. Aplicações Java para a arquitetura Cliente-Servidor possuem uma única aplicação Java Servidor, responsável por gerenciar as aplicações dos clientes e intermediar a comunicação entre eles pela rede.

III. Aplicações Java para a arquitetura Cliente-Servidor possuem sempre duas aplicações Java Cliente, cada uma delas conectada com a outra, sempre através do servidor, podendo utilizar tipos diferentes de dispositivos em rede.

IV. Um *socket* representa um canal de comunicação bidirecional entre duas aplicações, com um *socket* de um lado conectado ao *socket* do outro, possibilitando o tráfego de dados entre eles e entre suas aplicações.

É correto, apenas, o que se afirma em:

a) I e II;
b) I e III;
c) II e III;
d) II e IV;
e) III e IV.

<u>Questão 2</u>: (*1 ponto*)

Também conhecidos por *Software Design Patterns*, os Padrões de Projetos de *Software* devem ser estudados e utilizados para melhorar a comunicação da equipe, melhorar o aprendizado individual, aumentar a capacidade de correção e manutenção do código, fazer reuso das soluções anteriores e reduzir os recursos necessários para o desenvolvimento do projeto. Dentre os padrões mais adotados está o MVC – *Model-View-Controller*, que é um padrão de arquitetura de *software*.

Sendo assim, com base na referida bibliografia e no material de aula utilizados, avalie as afirmações a seguir:

I. Com o aumento da complexidade das aplicações, torna-se fundamental a separação entre os dados – *Model* (Modelo) e o controlador de aplicação – *View* (Visão);

II. Alterações feitas na interface com o usuário não devem afetar a manipulação de dados e os dados podem ser reorganizados sem alterar a interface com o usuário;

III. Para separar as tarefas de acesso aos dados e lógica de negócio, lógica de apresentação e de interação com o usuário, deve ser introduzido um componente entre os dois denominado *Controller* (Controlador); e

IV. O MVC define como os componentes da aplicação interagem e, apesar de ser um padrão de arquitetura de *software*, não é considerado um *software design pattern*.

É correto, apenas, o que se afirma em:

a) I e II;
b) I e III;
c) II e III;
d) II e IV;
e) III e IV.

<u>Questão 3</u>: (*1 ponto*)

Segundo os autores Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, no livro "*Design Patterns: Elements of Reusable Object-Oriented Software*", os padrões de projeto são soluções típicas para problemas recorrentes e comuns que ocorrem durante o desenvolvimento de *software*, tendo que os padrões não são modelos concluídos de projeto, mas fornecem esboços para como resolver problemas de projeto".

Levando-se em consideração esse contexto, a bibliografia adotada e o material de aula utilizado, avalie as seguintes asserções e a relação proposta entre elas:

I. Os padrões de projetos de *software* representam uma abordagem comprovada para se resolver problemas recorrentes no desenvolvimento de sistemas de *software* padronizados e confiáveis.

PORQUE

II. Os padrões de projetos de *software* são diretrizes e soluções de projetos amplamente aceitas, que ajudam a melhorar eficiência, qualidade e manutenção.

A respeito dessas asserções, assinale a única opção correta:

a) As asserções I e II são proposições falsas;

b) As asserções I e II são proposições verdadeiras e a II é uma justificativa da I;

c) As asserções I e II são proposições verdadeiras e a I é uma justificativa da II;

d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira;

e) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.

<u>Questão 4</u>: (*1 ponto*)

Stream, em Inglês, significa corrente ou fluxo e a partir do Java 8, seu conceito foi ampliado para se referir a um novo recurso da linguagem, utilizado para realizar o processamento dos dados de forma sequencial (serial) ou simultânea (paralela), podendo se referir ao fluxo de entrada e saída de dados, ou ao processamento de dados sequencial ou simultâneo.

Levando-se em consideração esse contexto, a bibliografia adotada e o material de aula utilizado, avalie as seguintes asserções e a relação proposta entre elas:

I. Os *streams* são os canais de comunicação entre o cliente e o servidor, ou seja, os *sockets*;

PORQUE

II. Um *stream* é usado em programação para permitir imprimir, ler e gravar dados de/em dispositivos e entrada e saída e, ao usá-lo, pode-se enviar ou receber os dados por meio de um fluxo.

A respeito dessas asserções, assinale a única opção correta:

a) As asserções I e II são proposições falsas;

b) As asserções I e II são proposições verdadeiras e a II é uma justificativa da I;

c) As asserções I e II são proposições verdadeiras e a I é uma justificativa da II;

d) As asserções I e II são proposições verdadeiras e não são justificativas uma da outra;

e) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.

Questão 5: (*6 pontos*)

Gerado por qualquer *IDE* fornecer o código fonte, em *Java*, das classes necessárias (incluindo a de execução), implementadas sob o paradigma da *Programação Orientada a Objetos*, além do programa ***.jar*** que as executa diretamente, objetivando resolver o problema descrito a seguir.

ATENÇÃO:

a) O aluno deverá fornecer uma lista de instruções detalhadas para o professor executar as aplicações desenvolvidas por ele;

b) Se o programa *.jar* fornecido não executar automaticamente a aplicação desenvolvida, a resolução da questão será invalidada;

c) As classes soluções desta questão (arquivos *.java*) deverão ser compactadas em um único arquivo (*.zip* ou *.rar*), em conjunto com o respectivo arquivo *.jar* funcional, além deste arquivo *.docx* da prova, contendo todas as respostas para as outras questões e a devida identificação do aluno e entregue em resposta à tarefa do *CanvasLMS* da prova.

Problemas:

I. Utilizando-se apenas os códigos vistos e praticados em aula e os códigos desenvolvidos pelo próprio aluno para esta disciplina, através de uma aplicação implementada durante a prova, deseja-se criar uma **comunicação bidirecional** entre **dois clientes**, e somente dois clientes, através de **um servidor**, do tipo *chat*. Toda comunicação entre os clientes deverá utilizar uma *UI - User Interface* (Interface com o Usuário) baseada em Janelas (*layout*, *swing* etc.) e não em console;

II. Utilizando-se apenas os códigos vistos e praticados em aula e os códigos desenvolvidos pelo próprio aluno para esta disciplina, através de uma aplicação implementada durante a prova, deseja-se que no **console do servidor** sejam apresentadas todas as mensagens relativas às conexões dos clientes (*status*) e, além disso, todas as mensagens trocadas entre os dois clientes conectados a ele, com devida identificação dos remetentes.

Casos de Teste (sequenciais):

1. Ao executar a **Instância 1 do Cliente**, antes de executar a **Instância 1 do Servidor**, o sistema deverá informar que houve problema, que não conseguiu permanecer com o respectivo aplicativo Cliente aberto, pois não há servidor disponível naquele endereço/porta para conexão;

2. Ao executar a **Instância 1 do Servidor**, o sistema deverá informar que não houve problema, que conseguiu executar o respectivo aplicativo Servidor e que aguarda pela conexão de até dois clientes ao mesmo tempo;

3. Ao executar a **Instância 2 do Servidor**, o sistema deverá informar que houve problema, que não conseguiu executar o respectivo aplicativo Servidor, pois já há um servidor em funcionamento naquele endereço/porta para conexão;

4. Ao executar a **Instância 1 do Cliente** novamente, agora depois de executar **Instância 1 do Servidor**, o sistema deverá informar que não houve problema, que está com o aplicativo Cliente aberto e conectado ao servidor e que está com a comunicação bidirecional com o servidor ativa;

5. Ao executar a **Instância 2 do Cliente**, depois de executar **Instância 1 do Servidor** e a **Instância 1 do Cliente**, o sistema deverá informar que não houve problema, que está com o aplicativo Cliente aberto e conectado ao servidor e que está com a comunicação bidirecional com o outro cliente ativa, aguardando pelas mensagens que serão trocadas entre eles;

6. Ao executar a **Instância 3 do Cliente**, depois de executar **Instância 1 do Servidor** e as **Instâncias 1 e 2 do Cliente**, o sistema deverá informar que houve problema, que não conseguiu permanecer com o respectivo aplicativo cliente aberto, pois já há duas instâncias de Cliente conectadas através do servidor;

7. Ao se digitar quaisquer mensagens na **Instância 1 do Cliente**, por comunicação bidirecional, essas mensagens deverão ser apresentadas na **Instância 2 do Cliente**, sem ecoar na Instância de origem;

8. Ao se digitar quaisquer mensagens na **Instância 2 do Cliente**, por comunicação bidirecional, essas mensagens deverão ser apresentadas na **Instância 1 do Cliente**, sem ecoar na Instância de origem.